

5

Deploying the Application with Docker and CI/CD

Now that we have successfully developed our first full-stack application with a backend service and a frontend, we are going to package our app into Docker images and learn how to deploy them using **continuous integration (CI)** and **continuous delivery (CD)** principles. We have already learned how to start Docker containers in [Chapter 2, Getting to Know Node.js and MongoDB](#). In this chapter, we will learn how to create our own Docker images to instantiate containers from. Then, we are going to manually deploy our application to a cloud provider. Finally, we are going to configure CI/CD to automate the deployment of our application. At the end of this chapter, we will have successfully deployed our first full-stack **MongoDB Express React Node.js (MERN)** application, and set it up for future automated deployments!

In this chapter, we are going to cover the following main topics:

- Creating Docker images
- Deploying our full-stack application to the cloud
- Configuring CI to automate testing
- Configuring CD to automate the deployment

Technical requirements

Before we start, please install all requirements from [Chapter 1, Preparing For Full-Stack Development](#), and [Chapter 2, Getting to Know Node.js and MongoDB](#).

The versions listed in those chapters are the ones used in the book. While installing a newer version should not be an issue, please note that certain steps might work differently on a newer version. If you are having an issue with the code and steps provided in this book, please try using the versions mentioned in [Chapter 1](#) and [Chapter 2](#).

You can find the code for this chapter on GitHub:

<https://github.com/PacktPublishing/Modern-Full-Stack-React-Projects/tree/main/ch5>.

The CiA video for this chapter can be found at:

<https://youtu.be/aQpIfCQGwew>

Creating Docker images

In [Chapter 2](#), *Getting to Know Node.js and MongoDB*, we learned that in the Docker platform, we use Docker images to create containers, which can then run services. We have already learned how to use the existing **mongo** image to create a container for our database service. In this section, we are going to learn how to create our own image to instantiate a container from. To do so, we first need to create a **Dockerfile**, which contains all the instructions needed to build the Docker image. First, we will create a Docker image for our backend service and run a container from it. Then, we will do the same for our frontend. Finally, we will create a **Docker Compose file** to start our database and backend services together with our frontend.

Creating the backend Dockerfile

A Dockerfile tells Docker step by step how to build the image. Each line in the file is an instruction telling Docker what to do. The format of a Dockerfile is as follows:

```
# comment  
INSTRUCTION arguments
```

Every Dockerfile must begin with a **FROM** instruction, which specifies which image the newly created image should be based on. You can extend your image from existing images, such as **ubuntu** or **node**.

Let's get started by creating the Dockerfile for our backend service:

1. Copy the **ch4** folder to a new **ch5** folder, as follows:

```
$ cp -R ch4 ch5
```

2. Create a new **backend/Dockerfile** file inside the **ch5** folder.
3. In this file, we first define a base image for our image, which will be version 20 of the **node** image:

```
FROM node:20
```

This image is provided by Docker Hub, similar to the **ubuntu** and **mongo** images we created containers from before.

NOTE

*Be careful to only use official images and images created by trusted authors. The **node** image, for example, is officially maintained by the Node.js team.*

4. Then, we set the working directory, which is where all files of our service will be placed inside the image:

```
WORKDIR /app
```

The **WORKDIR** instruction is similar to using **cd** in the terminal. It changes the working directory so that we do not have to prefix all the

following commands with the full path. Docker creates the folder for us if it does not exist yet.

5. Next, we copy the **package.json** and **package-lock.json** files from our project to the working directory:

```
COPY package.json package-lock.json ./
```

The **COPY** instruction copies files from your local file system into the Docker image (relative to the local working directory). Multiple files can be specified, and the last argument to the instruction is the destination (in this case, the current working directory of the image).

The **package-lock.json** file is needed to ensure that the Docker image contains the same versions of the **npm** packages as our local build.

6. Now, we run **npm install** to install all dependencies in the image:

```
RUN npm install
```

The **RUN** instruction executes a command in the working directory of the image.

7. Then, we copy the rest of our application from the local file system to the Docker image:

```
COPY . .
```

NOTE

*Are you wondering why we initially just copied **package.json** and **package-lock.json**? Docker images are built layer by layer. Each instruction forms a layer of the image. If something changes, only the layers following the change are rebuilt. So, in our case, if any of the code changes, only this last **COPY** instruction is re-executed when rebuilding the Docker image. Only if dependencies change are the other **COPY** instruction and **npm install** re-executed. Using this order of instruction reduces the time required to rebuild the image immensely.*

8. Finally, we run our application:

```
CMD ["npm", "start"]
```

The **CMD** instruction is not executed while building the image. Instead, it stores information in the metadata of the image, telling Docker which command to run when a container is instantiated from the image. In our case, the container is going to run **npm start** when using our image.

NOTE

*You may have noticed that we passed a JSON array to the **CMD** instruction instead of simply writing **CMD npm start**. The JSON array version is called **exec form** and, if the first argument is an executable, will run the command directly without invoking a shell. The form without the JSON array is called **shell form** and will execute the command with a shell, prefixing it with **/bin/sh -c**. Running a command without a shell has the advantage of*

allowing the application to properly receive signals, such as a **SIGTERM** or **SIGKILL** signal when the application is terminated. Alternatively, the **ENTRYPOINT** instruction can be used to specify which executable should be used to run a certain command (it defaults to **/bin/sh -c**). In some cases, you may even want to run the script directly using **CMD ["node", "src/index.js"]**, so that the script can properly receive all signals. However, this would require us to implement the **SIGINT** signal in our back-end server to allow closing the container via **Ctrl + C**, so, to keep things simple, we just use **npm start** instead.

After creating our Dockerfile, we should also create a **.dockerignore** file to make sure unnecessary files are not copied into our image.

Creating a .dockerignore file

The **COPY** command, where we copy all files, would also copy the **node_modules** folder and other files, such as the **.env** file, which we do not want to go into our image. To prevent certain files from being copied into our Docker image, we need to create a **.dockerignore** file. Let's do that now:

1. Create a new **backend/.dockerignore** file.
2. Open it and enter the following contents to ignore the **node_modules** folder and all **.env** files:

```
node_modules
.env*
```

Now that we have defined a **.dockerignore** file, the **COPY** instructions will ignore these folders and files. Let's build the Docker image now.

Building the Docker image

After successfully creating the backend Dockerfile and a **.dockerignore** file to prevent certain files and folders from being added to our Docker image, we can now get started building our Docker image:

1. Open a Terminal.
2. Run the following command to build the Docker image:

```
$ docker image build -t blog-backend backend/
```

We specified **blog-backend** as the name of our image and **backend/** as the working directory.

After running the command, Docker will start by reading the Dockerfile and **.dockerignore** file. Then, it will download the **node** image and run our instructions one by one. Finally, it will export all layers and metadata into our Docker image.

The following screenshot shows the output of creating a Docker image:

```

$ docker image build -t blog-backend backend/
[+] Building 120.8s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 159B
=> [internal] load .dockerignore
=> => transferring context: 59B
=> [internal] load metadata for docker.io/library/node:latest
=> [auth] library/node:pull token for registry-1.docker.io
=> [1/5] FROM docker.io/library/node:latest@sha256:db2672e3c200b85e0b813cdb294fac16764711d7a66b41315e6261f2231f2331
=> resolve docker.io/library/node:latest@sha256:db2672e3c200b85e0b813cdb294fac16764711d7a66b41315e6261f2231f2331
=> sha256:05962e2d2a19f217fca55ed6b61aae16070c7c220c690aee1b6056ae03fb377 7.54kB / 7.54kB
=> sha256:df2021ddb7d686b0bb125598b2a6163d63835f008356b3014595f354ea0b4006 49.61MB / 49.61MB
=> sha256:8d647f1dd7e741209a8a75083ccc889e39c3e94c17f45441eae96a1a679d971 23.50MB / 23.50MB
=> sha256:db2672e3c200b85e0b813cdb294fac16764711d7a66b41315e6261f2231f2331 1.21kB / 1.21kB
=> sha256:63db8a1a9663e2daa5fa8f49bce6b048cc5e31565f9786b1eca7d8be3f911ec5 2.00kB / 2.00kB
=> sha256:5cdd9a70365f741a6b9f7a4e32c0b7d4aa29ac73da0b78ca0a83e937f285fdd5 63.99MB / 63.99MB
=> sha256:95089c600b361807380090316c250b0b8eaf4fa2175b11ac8f49bb7581c61125 202.45MB / 202.45MB
=> sha256:00e0658e345c743de5051c897a98b802ae91e6375952104fafbf7d969ab581 3.37kB / 3.37kB
=> sha256:95f3e0ec3833596867c7796e2860de73800b5599a63bec578722cd6e857b3f 49.00MB / 49.00MB
=> extracting sha256:df2021ddb7d686b0bb125598b2a6163d63835f008356b3014595f354ea0b4006
=> sha256:335e1806199ff1916aeece3a5571fcd0012f56e3abfddbdf0df6e8a859eef 2.23MB / 2.23MB
=> sha256:764c4860e9c44928a589ad3af023dddb3cfff4b1bddff7c8377393e57fd47db9 452B / 452B
=> extracting sha256:8d647f1dd7e741209a8a75083ccc889e39c3e94c17f45441eae96a1a679d971
=> sha256:5cdd9a70365f741a6b9f7a4e32c0b7d4aa29ac73da0b78ca0a83e937f285fdd5
=> extracting sha256:95089c600b361807380090316c250b0b8eaf4fa2175b11ac8f49bb7581c61125
=> extracting sha256:00e0658e345c743de5051c897a98b802ae91e6375952104fafbf7d969ab581
=> extracting sha256:95f3e0ec3833596867c7796e2860de73800b5599a63bec578722cd6e857b3f
=> extracting sha256:764c4860e9c44928a589ad3af023dddb3cfff4b1bddff7c8377393e57fd47db9
=> [internal] load build context
=> transferring context: 297.81kB
=> [2/5] WORKDIR /app
=> [3/5] COPY package.json package-lock.json ./
=> [4/5] RUN npm install
=> [5/5] COPY . .
=> exporting to image
=> exporting layers
=> writing image sha256:df44d3116a1f3adf1445f2aed7fd220e45083da7388ce6ffa8699a7e80babaf0
=> naming to docker.io/library/blog-backend

```

Figure 5.1 – The output when creating a Docker image

Now that we have successfully created our own image, let's create and run a container based on it!

Creating and running a container from our image

We have already created Docker containers based on the **ubuntu** and **mongo** images in [Chapter 2. Getting to Know Node.js and MongoDB](#). Now, we are going to create and run a container from our own image. Let's get started doing that now:

1. Run the following command to list all available images:

```
$ docker images
```

This command should return the **blog-backend** image that we just created, and the **mongo** and **ubuntu** images that we previously used.

2. Make sure the **dbserver** container with our database is already running.
3. Then, start a new container, as follows:

```
$ docker run -it -e PORT=3001 -e DATABASE_URL=mongodb://host.docker.internal:27017/blog -p 3001
```

Let's break down the arguments to the **docker run** command:

1. **-it** runs the container in interactive mode (**-t** to allocate a pseudo Terminal and **-i** to keep the input stream open).
2. **-e PORT=3001** sets the **PORT** environment variable inside the container to **3001**.
3. **-e DATABASE_URL=mongodb://host.docker.internal:27017/blog** sets the **DATABASE_URL** environment variable. Here, we replaced **localhost** with **host.docker.internal**, as the MongoDB service runs in a different container on the Docker host (our machine).
4. **-p 3001:3001** forwards port **3001** from inside the container to port **3001** on the host (our machine).
5. **blog-backend** is the name of our image.

4. The **blog-backend** container is now running, which looks very similar to running the backend directly on our host in the Terminal. Go to **http://localhost:3001/api/v1/posts** to verify that it is running properly like before and returning all posts.
5. Keep the container running for now.

We have successfully packaged our backend as a Docker image and started a container from it! Now, let's do the same for our frontend.

Creating the frontend Dockerfile

After creating a Docker image for the backend service, we are now going to repeat the same process to create an image for the frontend. We will do so by first creating a Dockerfile, then the **.dockerignore** file, building the image, and then running a container. Now, we will start with creating the frontend Dockerfile.

In the Dockerfile for our frontend, we are going to use two images:

- A **build** image to build our project using **Vite** (which will be discarded, with only the build output kept)
- A **final** image, which will serve our static site using **nginx**

Let's make the Dockerfile now:

1. Create a new Dockerfile in the root of our project.
2. In this newly created file, first, use the **node** image again, but this time we tag it **AS build**. Doing so enables multi-stage builds in Docker, which means that we can use another base image later for our **final** image:

```
FROM node:20 AS build
```

3. During build time, we also set the **VITE_BACKEND_URL** environment variable. In Docker, we can use the **ARG** instruction to define environment variables that are only relevant when the image is being built:

```
ARG VITE_BACKEND_URL=http://localhost:3001/api/v1
```

NOTE

While the **ARG** instruction defines an environment variable that can be changed at build time using the **--build-arg** flag, the **ENV** instruction sets the environment variable to a fixed value, which will persist when a container is run from the resulting image. So, if we want to customize environment variables during build time, we should use the **ARG** instruction. However, if we want to customize environment variables during runtime, **ENV** is better suited.

4. We set the working directory to **/build** for the **build** stage, and then repeat the same instructions that we defined for the backend to install all necessary dependencies and copy over the necessary files:

```
WORKDIR /build
COPY package.json .
COPY package-lock.json .
RUN npm install
COPY . .
```

5. Additionally, we execute **npm run build** to create a static build of our Vite app:

```
RUN npm run build
```

6. Now, our **build** stage is completed. We use the **FROM** instruction again to create the **final** stage. This time, we base it off the **nginx** image, which runs an nginx web server:

```
FROM nginx AS final
```

7. We set the working directory for this stage to **/var/www/html**, which is the folder that nginx serves static files from:

```
WORKDIR /usr/share/nginx/html
```

8. Lastly, we copy everything from the **/build/dist** folder (which is where Vite puts the built static files) from the **build** stage into the **final** stage:

```
COPY --from=build /build/dist .
```

A **CMD** instruction is not needed in this case, as the **nginx** image already contains one to run the web server properly.

We successfully created a multi-stage Dockerfile for our frontend! Now, let's move on to creating the **.dockerignore** file.

Creating the **.dockerignore** file for the frontend

We also need to create a **.dockerignore** file for the frontend. Here, we also exclude, in addition to the **node_modules/** folder and **.env** files, the **backend/** folder containing our backend service and the **.vscode**, **.git**, and **.husky** folders. Let's create the **.dockerignore** file now:

1. Create a new **.dockerignore** file in the root of our project.
2. Inside this newly created file, enter the following contents:

```
node_modules
.env*
backend
.vscode
.git
.husky
.commitlintrc.json
```

Now that we have ignored the files not necessary for the Docker image, let's build it!

Building the frontend Docker image

Just like before, we execute the **docker build** command to build the image, giving it the name **blog-frontend** and specifying the root directory as the path:

```
$ docker build -t blog-frontend .
```

Docker will now use the **node** image to build our frontend in the **build** stage. Then, it will switch to the **final** stage, use the **nginx** image, and copy over the built static files from the **build** stage.

Now, let's create and run the frontend container.

Creating and running the frontend container

Similarly to what we did for the backend container, we can also create and run a container from the **blog-frontend** image by executing the following command:

```
$ docker run -it -p 3000:80 blog-frontend
```

The **nginx** image runs the web server on port **80**, so, if we want to use the port **3000** on our host, we need to forward from port **80** to **3000** by passing **-p 3000:80**.

After running this command and navigating to **http://localhost:3000** in your browser, you should see the frontend being served properly and showing blog posts from the backend.

Now that we have created images and containers for the backend and frontend, we are going to learn about a way to manage multiple images more easily.

Managing multiple images using Docker Compose

Docker Compose is a tool that allows us to define and run multi-container applications with Docker. Instead of manually building and running the backend, frontend, and database containers, we can use Compose to build and run them all together. To get started using Compose, we need to create a **compose.yaml** file in the root of our project, as follows:

1. Create a new **compose.yaml** file in the root of our project.
2. Open the newly created file and start by defining the version of the Docker Compose file specification:

```
version: '3.9'
```

3. Now, define a **services** object, in which we are going to define all the services that we want to use:

```
services:
```

4. First, we have **blog-database**, which uses the **mongo** image and forwards port **27017**:

```
blog-database:
  image: mongo
  ports:
    - '27017:27017'
```

NOTE

In YAML files, the indentation of lines is very important to distinguish where properties are nested, so please be careful to put in the correct amount of spaces before each line.

5. Next, we have **blog-backend**, which uses the Dockerfile defined in the **backend/** folder, defines the environment variables for **PORT** and **DATABASE_URL**, forwards the port to the host, and depends on **blog-database**:

```
blog-backend:
  build: backend/
  environment:
    - PORT=3001
    - DATABASE_URL=mongodb://host.docker.internal:27017/blog
  ports:
    - '3001:3001'
  depends_on:
    - blog-database
```

6. Lastly, we have **blog-frontend**, which uses the Dockerfile defined in the root, defines the **VITE_BACKEND_URL** build argument, forwards the port to the host, and depends on **blog-backend**:

```
blog-frontend:
  build:
    context: .
    args:
      VITE_BACKEND_URL: http://localhost:3001/api/v1
  ports:
    - '3000:80'
  depends_on:
    - blog-backend
```

7. After defining the services, save the file.
8. Then, stop the backend and frontend containers running in the terminal by using the **Ctrl + C** key combination.
9. Also, stop the already running **dbserver** container, as follows:

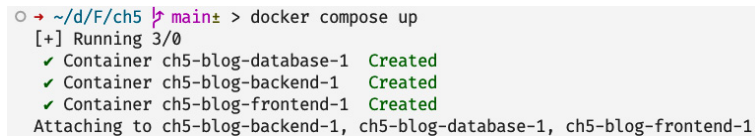
```
$ docker stop dbserver
```

10. Finally, run the following command in the Terminal to start all services using Docker Compose:

```
$ docker compose up
```

Docker Compose will now create containers for the database, backend, and frontend and start all of them. You will start seeing logs being printed from the different services. If you go to <http://localhost:3000>, you can see that the frontend is running. Create a new post to verify that the connection to the backend and database works as well.

The following screenshot shows the output of `docker compose up` creating and starting all containers:



```
○ → ~/d/F/ch5 main: > docker compose up
[+] Running 3/0
 ✓ Container ch5-blog-database-1 Created
 ✓ Container ch5-blog-backend-1 Created
 ✓ Container ch5-blog-frontend-1 Created
Attaching to ch5-blog-backend-1, ch5-blog-database-1, ch5-blog-frontend-1
```

Figure 5.2 – Creating and running multiple containers with Docker Compose

The output in the screenshot is then followed by log messages from the various services, including the MongoDB database service and our backend and frontend services.

Just like always, you can press `Ctrl + C` to stop all Docker Compose containers.

Now that we have set up Docker Compose, it's very easy to start all services at once and manage them all in one place. If you look at your Docker containers, you may notice that there are lots of stale containers still left over from previously building the **blog-backend** and **blog-frontend** containers. Let's now learn how to clean up those.

Cleaning up unused containers

After experimenting with Docker for a while, there will be lots of images and containers that are not in use anymore. Docker generally does not remove objects unless you explicitly ask it to, causing it to use a lot of disk space. If you want to remove objects, you can either remove them one by one or use one of the **prune** commands provided by Docker:

- **docker container prune**: This removes all stopped containers
- **docker image prune**: This removes all dangling images (images not tagged and not referenced by any container)
- **docker image prune -a**: This removes all images not used by any containers
- **docker volume prune**: This removes all volumes not used by any containers

- **docker network prune**: This cleans up networks not used by any containers
- **docker system prune**: This prunes everything except volumes
- **docker system prune --volumes**: This prunes everything

So, if you want to, for example, remove all unused containers, you should first make sure that all of the containers that you still want to use are running. Then, execute **docker container prune** in the terminal.

Now that we have learned how to use Docker locally to package our services as images and run them in containers, let's move on to deploying our full-stack application to the cloud.

Deploying our full-stack application to the cloud

After creating Docker images and containers locally, it's time to learn how to deploy them to the cloud so that everyone can access our services. In this book, we are going to use **Google Cloud** as an example, but the general process also applies to other providers such as **Amazon Web Services (AWS)** and **Microsoft Azure**. For the MongoDB database, we are going to use **MongoDB Atlas** but feel free to use any provider that can host a MongoDB database for you.

Creating a MongoDB Atlas database

To host our database, we are going to use the official cloud solution provided by the MongoDB team called MongoDB Atlas. Let's get started with registering and setting up a database now:

1. Go to <https://www.mongodb.com/atlas> and press **Try Free** to create a new account, or sign in with your existing account.

NOTE

The following instructions may vary slightly due to updates in the MongoDB Atlas UI. If the options are not available exactly as listed, try to follow the instructions on the website instead to create a database and a user to access it. This applies to all cloud services that we are going to set up throughout this chapter.

2. Select **Database** from the sidebar, then press **Create** to create a new database deployment. If you made a new account, you should be asked to create a new database deployment automatically.
3. Select **Shared / M0 Sandbox** (free instance) on Google Cloud and your preferred region.
4. Give your cluster a name of your choice.
5. Press **Create** to create your M0 sandbox cluster. It will take some time for the database to be accessible (typically around a minute).
However, you can continue setting up the user while waiting for the cluster to be set up.

- Go to the **Database** section in the sidebar and click on the **Connect** button next to your newly created cluster.
- In the popup, select **Allow Access from Anywhere** and then press **Add IP Address**.
- Set a username and password for your database user and press **Create database user**.
- Press **Choose a connection method** and select **Drivers**.
- A connection string will be shown; copy it and save it for later, inserting your previously set password instead of the `<password>` string. The connection string should have the following format:

```
mongodb+srv://<username>:<password>@<cluster-name>.<cluster-id>.mongodb.net/?retryWrites=true&w
```

- Verify that the connection string works by opening a terminal and connecting to it using **mongo** shell:

```
$ mongosh "<connection-string>"
```

The following screenshot shows how the **Database Deployments** tab looks in MongoDB Atlas:

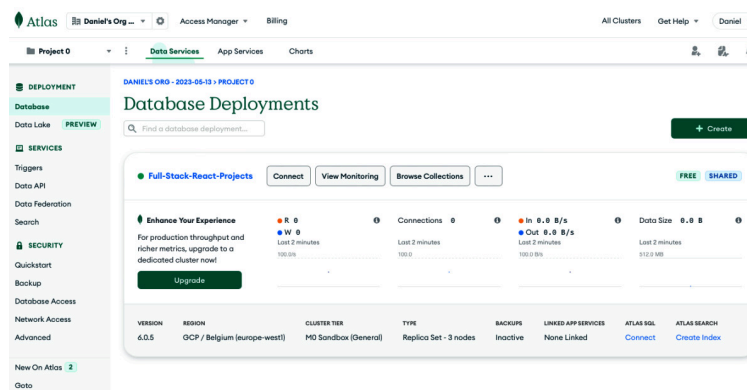


Figure 5.3 – A fresh M0 Sandbox database cluster deployed on MongoDB Atlas

Now that we have successfully created our MongoDB database in the cloud, we can move on to setting up Google Cloud to deploy our backend and frontend.

Creating an account on Google Cloud

Let's get started with Google Cloud by creating an account now. When creating an account, you need to enter billing information, but you will get \$300 in free credits to trial using Google Cloud for free:

- Go to <https://cloud.google.com> in your browser.
- Press **Get started for free** if you do not have an account yet or press **Sign in** if you already have an account.
- Log in with your Google account and follow the instructions until you have access to the Google Cloud console.

You should now see a screen similar to the following figure:

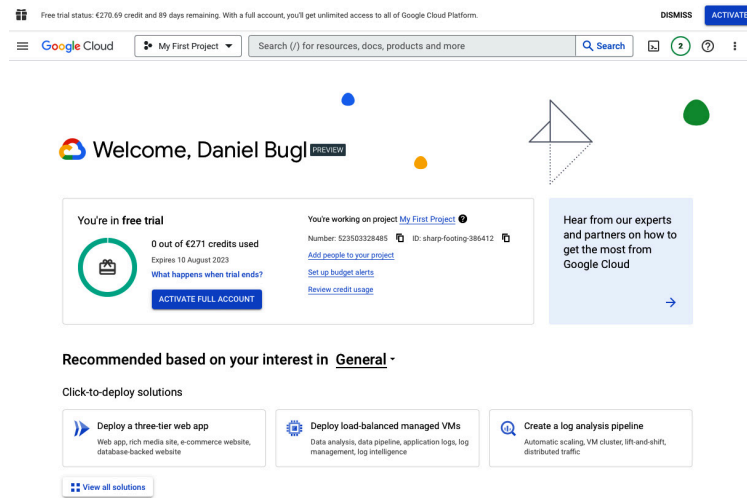


Figure 5.4 – The Google Cloud console after registering

Now that you have an account set up and ready, let's start deploying our services.

Deploying our Docker images to a Docker registry

Before we can deploy a service on a cloud provider, we first need to deploy our Docker image to a **Docker registry** so that the cloud provider can access it from there and create a container from it. Follow these steps to deploy our Docker images to Docker Hub, the official Docker registry:

1. Go to <https://hub.docker.com> and log in or register an account there.
2. Press the **Create repository** button to create a new repository. The repository will contain our image.
3. Enter **blog-frontend** as the repository name and leave the description empty and visibility **public**. Then press the **Create** button.
4. Repeat *Steps* 2 and 3, but this time, enter **blog-backend** as the repository name.
5. Open a new terminal and enter the following command to log in to your Docker Hub account:

```
$ docker login
```

Enter your username and password from Docker Hub and press the *Return* key or *Enter*.

6. Rebuild your image for Linux (to be able to deploy it to Google Cloud later), tag your image with your repository name (replace **[USERNAME]** with your Docker Hub username), and push it to the repository:

```
$ docker build --platform linux/amd64 -t blog-frontend .
$ docker tag blog-frontend [USERNAME]/blog-frontend
$ docker push [USERNAME]/blog-frontend
```

7. Navigate to **backend/** in the terminal and repeat *Step 6* for the **blog-backend** image:

```
$ cd backend/
$ docker build --platform linux/amd64 -t blog-backend .
```

```
$ docker tag blog-backend [USERNAME]/blog-backend
$ docker push [USERNAME]/blog-backend
```

Now that both repositories are set up and the images are pushed to them, they should show up in Docker Hub with the following information:

Contains: Image | Last pushed: a few seconds ago:

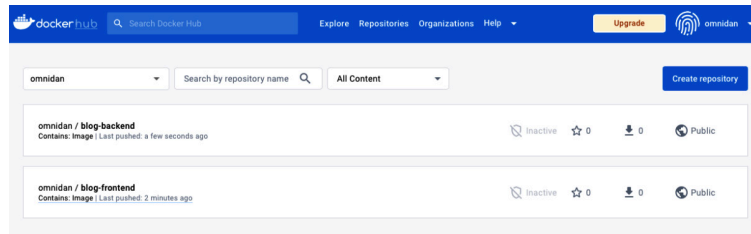


Figure 5.5 – Docker Hub giving an overview of our repositories

Now that our Docker images are published on a public Docker registry (Docker Hub), we can continue setting up Google Cloud to deploy our services.

NOTE

*The repositories created on Docker Hub in this book are public. You can also choose to create up to one private repository on Docker Hub for free. Otherwise, you either need to have a Docker Hub subscription, use a different registry, or host your own registry. For example, **Google Artifact Registry** could be used to deploy private Docker images on **Cloud Run**.*

Deploying the backend Docker image to Cloud Run

After successfully publishing our Docker images on the Docker Hub registry, it's time to deploy them using Google Cloud Run. Cloud Run is a managed compute platform. It allows us to run containers directly on the Google Cloud infrastructure, making app deployment simple and fast. The alternatives to Cloud Run would be Kubernetes-based infrastructure, such as AWS ECS Fargate or DigitalOcean.

Follow these steps to deploy the backend to Google Cloud Run:

1. Go to <https://console.cloud.google.com/>.
2. In the search bar at the top, enter **Cloud Run** and select the **Cloud Run – Serverless for containerized applications** product.
3. Press the **Create Service** button to create a new service.

NOTE

*You may need to first create a project before you can create a service. In that case, just follow the instructions on the website to create a new project with a name of your choice. Afterward, press the **Create Service** button to create a new service.*

4. Enter `[USERNAME]/blog-backend` in the **Container image URL** box.

5. Enter **blog-backend** in the **Service name** box, select a region of your choice, leave **CPU is only allocated during request processing** selected, and select **All – Allow direct access to your service from the Internet** and **Authentication – Allow unauthenticated invocations**.
6. Expand the **Container, Networking, Security** section, scroll down to **Environment variables**, and click on **Add Variable**.
7. Name the new environment variable **DATABASE_URL** and, as the value, enter the connection string from MongoDB Atlas, which you saved earlier.

NOTE

*For simplicity, we are using a regular environment variable here. To make variables that contain credentials more secure, it should instead be added as a secret, which requires enabling the **Secrets API**, adding the secret to the secret manager, and then referencing the secret and choosing it to be exposed as an environment variable.*

8. Leave the rest of the options as the default options and press **Create**.
9. You will get redirected to the newly created service, where the container is currently being deployed. Wait until it finishes deploying, which can take up to a couple of minutes.
10. When the service finishes deploying, you should see a checkmark and a URL. Click the URL to open the backend and you will see our **Hello World from Express!** message, which means that our backend was successfully deployed in the cloud!

A deployed service looks as follows in Google Cloud Run:

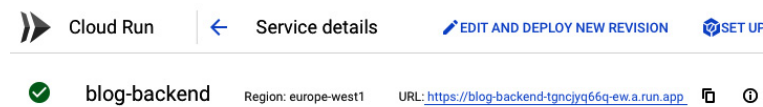


Figure 5.6 – A successfully deployed service on Google Cloud Run

Deploying the frontend Docker image to Cloud Run

For the frontend, we first need to rebuild the container to change the **VITE_BACKEND_URL** environment variable, which is statically built into our project. Let's do that first:

1. Open a terminal and run the following command to rebuild the frontend with the environment variable set:

```
$ docker build --platform linux/amd64 --build-arg "VITE_BACKEND_URL=[URL]/api/v1" -t blog-front
```

Make sure to replace **[URL]** with the URL to the backend service deployed on Google Cloud Run.

2. Tag it with your Docker Hub username and deploy the new version of the image to Docker Hub:

```
$ docker tag blog-frontend [USERNAME]/blog-frontend
```

```
$ docker push [USERNAME]/blog-frontend
```

Now, we can repeat similar steps as we did to deploy the backend to deploy our frontend as well:

1. Create a new Cloud Run service, enter `[USERNAME]/blog-frontend` in the **Container image URL** box and **blog-frontend** in the **Service name** box.
2. Pick a region of your choice and enable **Allow unauthenticated invocations**.
3. Expand **Container**, **Networking**, **Security** and change the container port from **8080** to **80**.
4. Press **Create** to create the service and wait for it to be deployed.
5. Open the URL in your browser and you should see the deployed frontend. Adding and listing blog posts also works now by sending a request to the deployed backend, which then stores the posts in our MongoDB Atlas cluster.

We have successfully manually deployed our first full-stack React and Node.js application with a MongoDB database in the cloud! In the next sections, we are going to focus on automating testing and deployment using CI/CD.

Configuring CI to automate testing

Continuous Integration (CI) covers the automation of integrating code changes to find bugs quicker and keep the code base easily maintainable. Usually, this is facilitated by having scripts run automatically when a developer makes a pull/merge request before the code is merged into the main branch. This practice allows us to detect problems with our code early by, for example, running the linter and tests before the code can be merged. As a result, CI gives us more confidence in our code and allows us to make and deploy changes faster and more frequently.

The following figure shows a simple overview of a possible CI/CD pipeline:

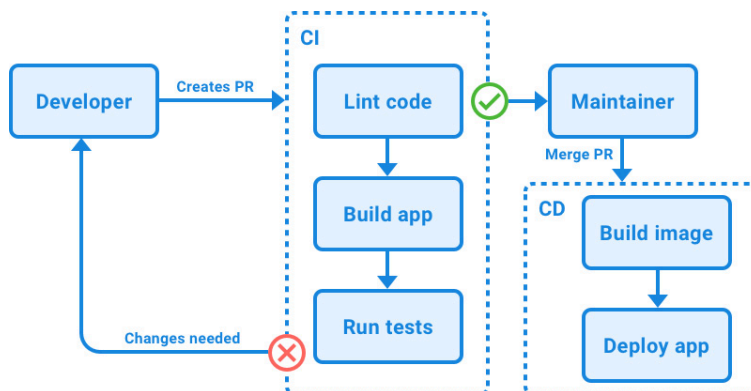


Figure 5.7 – Simple overview of a CI/CD pipeline

NOTE

*In this book, we are going to use **GitHub Actions** for CI/CD. While the syntax and configuration files might look and work differently on other systems, such as **GitLab CI/CD** or **CircleCI**, the general principles are similar.*

In GitHub Actions, **workflows** can be triggered when **events** occur in the repository, such as pushing to a branch, opening a new pull request, or creating a new issue. Workflows can contain one or multiple **jobs**, which can either run in parallel or sequentially. Each job runs inside its own **runner**, which takes instructions from the CI definition and executes them within a specified container. Inside jobs, **actions** can be performed, which are either existing actions provided on GitHub, or we can write our own actions.

Adding CI for the frontend

Let's get started creating a workflow that will build the frontend when a pull request is created, or a push is made to the **main** branch:

1. Create a new **.github/** folder in the root of our project. Inside it, create a **workflows/** folder.
2. Inside the **.github/workflows/** folder, create a new file called **frontend-ci.yaml**.
3. Open the **.github/workflows/frontend-ci.yaml** file and start by giving the workflow a name:

```
name: Blog Frontend CI
```

4. Then, listen to events by using the **on** keyword. We are going to execute the jobs when a new pull request or push is made to the **main** branch:

```
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
```

5. Now, we define a job that will run the linter and build the frontend:

```
jobs:
  lint-and-build:
```

6. We run the job on an **ubuntu-latest** container:

```
runs-on: ubuntu-latest
```

7. We can make use of the matrix strategy to run our tests multiple times with different variables. In our case, we want to run it on multiple Node.js versions:

```
strategy:
  matrix:
    node-version: [16.x, 18.x, 20.x]
```

8. Now, we define the steps inside our job. Make sure the **steps** are defined on the same indentation level as **strategy**:

```
steps:
```

9. First, we use the **actions/checkout** action, which checks out our repository:

```
- uses: actions/checkout@v3
```

10. Then, we use the **actions/setup-node** action, which sets up Node.js inside our container. Here, we make use of the **node-version** variable we defined earlier:

```
- name: Use Node.js ${ matrix.node-version }
  uses: actions/setup-node@v3
  with:
    node-version: ${ matrix.node-version }
    cache: 'npm'
```

The **cache** option specifies a package manager to be used for caching dependencies.

11. Finally, we install dependencies, run the linter, and build our frontend:

```
- name: Install dependencies
  run: npm install
- name: Run linter on frontend
  run: npm run lint
- name: Build frontend
  run: npm run build
```

Adding CI for the backend

Now that we have added CI for the frontend, let's also add CI for the backend by building and testing it when a pull request is created or a push is made to the **main** branch:

1. Inside the **.github/workflows/** folder, create a new file called **backend-ci.yaml**.
2. Open the **.github/workflows/backend-ci.yaml** file, start by giving it a name, and listen to the same events as we did for the frontend CI:

```
name: Blog Backend CI
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
```

3. Now, we define a job that will build and test the backend. We set the default working directory to the **backend/** folder to run all actions inside that folder:

```

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [16.x, 18.x, 20.x]
    defaults:
      run:
        working-directory: ./backend

```

4. Then, we use the same actions as for the frontend to check out the repository and set up Node.js:

```

steps:
  - uses: actions/checkout@v3
  - name: Use Node.js ${ matrix.node-version }
    uses: actions/setup-node@v3
    with:
      node-version: ${ matrix.node-version }
      cache: 'npm'
  - name: Install dependencies
    run: npm install

```

5. Finally, we run the linter on our backend and run the tests:

```

- name: Run linter on backend
  run: npm run lint
- name: Run backend tests
  run: npm test

```

6. Save the workflow files and commit and push them to a GitHub repository by creating a new repository on GitHub and following their instructions to push an existing repository to GitHub.
7. Go to the repository on GitHub and select the **Actions** tab. You should see your workflows running here.

The following screenshot shows our CI workflows successfully running on GitHub:

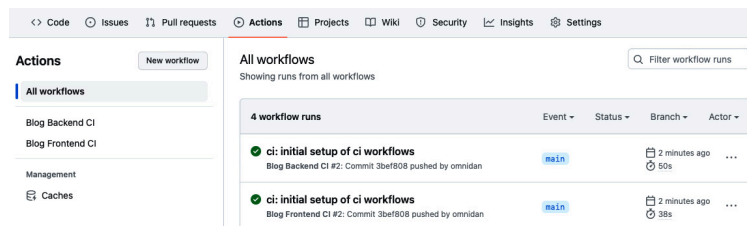


Figure 5.8 – Backend and frontend CI workflows successfully running in GitHub Actions

If we make a new pull request to the **main** branch, we can also see that our CI workflows are running properly on the new code. For example, if we added a way to tag posts from the frontend and accidentally made tags required in the backend without considering our previous rule of only the title being required, we will see that the corresponding tests failed:

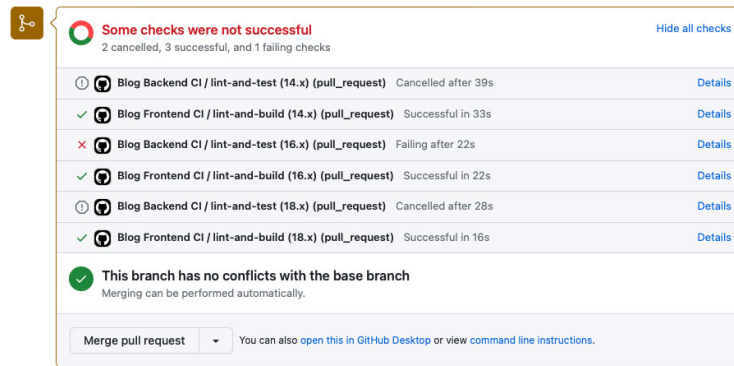


Figure 5.9 – Backend CI workflow failing in a pull request

We can also see that GitHub Actions automatically cancels the jobs running for other Node.js versions after one of them already failed, to avoid wasting time.

Now that we have successfully set up our CI workflows, let's continue by setting up CD to automate the deployment of our full-stack application.

Configuring CD to automate the deployment

After the pull/merge request is merged, **continuous delivery (CD)** comes into play. CD automates the release process by automatically deploying the services and applications for us. Usually, this involves a multi-stage process, where code is first automatically deployed to a staging environment and can then be manually deployed to other environments, up until production. If deployment to production is also an automated process, it is called **continuous deployment** instead of continuous delivery.

First, we need to get the credentials to authenticate with Docker Hub and Google Cloud. Then, we can set up the workflow for deploying our blog.

Getting Docker Hub credentials

Let's start by getting the credentials to access Docker Hub:

1. Go to <https://hub.docker.com/>.
2. Click on your profile and go to your account settings.
3. Click on the **Security** tab and press the **New Access Token** button.
4. As a description, write **GitHub Actions** and press the **Generate** button. Give **Read, Write, Delete** permissions.
5. Copy the access token and store it in a safe place.
6. Go to your GitHub repository and then go to **Settings | Secrets and variables | Actions**.
7. Press the **New repository secret** button to add a new secret. As a name, write **DOCKERHUB_USERNAME**, and as a secret value, use your username on Docker Hub.
8. Add another secret with the name **DOCKERHUB_TOKEN** and paste your previously created access token as the secret value.

Getting Google Cloud credentials

Now, we are going to create a service account to access Google Cloud Run:

1. Go to <https://console.cloud.google.com/>.
2. In the search box on the top, enter **Service accounts** and go to the **IAM and admin – Service accounts** page.
3. Press the **Create Service Account** button.
4. In the **Service account name** box, enter **GitHub Actions**. The ID should automatically be generated as **github-actions**. Press **Create and Continue**.
5. Grant the service access to the **Cloud Run Admin** role and press **Continue**.
6. Press **Done** to finish creating the service account.
7. On the overview list, copy the email of your newly created service account and save it for later use.
8. Go to the default compute service account by clicking on its email address. Go to the **Permissions** tab and press **Grant Access**.
9. Paste the email of your newly created service account into the **New principals** field and assign the **Cloud Run Service Agent** role. Press **Save** to confirm.
10. On the overview list, press the three dots icon to open actions on your **github-actions** service account and select **Manage keys**.
11. On the new page, press **Add Key | Create New Key**, and press **Create** in the popup. A JSON file should be downloaded.
12. Go to your GitHub repository, and go to **Settings | Secrets and variables | Actions**. Press the **New repository secret** button to add a new secret.
13. Add a new secret on your GitHub repository called **GOOGLECLOUD_SERVICE_ACCOUNT** and paste the previously copied email of your newly created service account as a secret value.
14. Add a new secret on your GitHub repository called **GOOGLECLOUD_CREDENTIALS** and as the secret, paste in the contents of the downloaded JSON file.
15. Add a new secret on your GitHub repository called **GOOGLECLOUD_REGION** and set the secret value to the region you selected when creating the Cloud Run services.

NOTE

For better security, Google recommends using **workload identity federation** instead of exporting service account key JSON credentials. However, setting up workload identity federation is a bit more complicated. More information on how to set it up can be found here:

<https://github.com/google-github-actions/auth#setup>.

Defining the deployment workflow

Now that the credentials are available as secret values to our CI/CD workflows, we can get started defining the deployment workflow:

1. Inside the `.github/workflows/` folder, create a new file called `cd.yaml`.
2. Open the `.github/workflows/cd.yaml` file and start by giving it a name:

```
name: Deploy Blog Application
```

3. For CD, we only execute the workflow when pushing to the `main` branch:

```
on:
  push:
    branches:
      - main
```

4. We start defining a `deploy` job, in which we set `environment` to `production` and point the URL to the deployed frontend URL:

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: production
      url: ${ steps.deploy-frontend.outputs.url }
```

We will define a step with the `deploy-frontend` ID later, which stores a variable in `steps.deploy-frontend.outputs.url`.

5. For the steps, as we did before, we first need to check out our repository:

```
steps:
  - uses: actions/checkout@v3
```

6. Then, we log in to Docker Hub using the credentials we set earlier in our secrets:

```
- name: Login to Docker Hub
  uses: docker/login-action@v2
  with:
    username: ${ secrets.DOCKERHUB_USERNAME }
    password: ${ secrets.DOCKERHUB_TOKEN }
```

7. Next, we log in to Google Cloud using the credentials we set earlier:

```
- uses: google-github-actions/auth@v1
  with:
    service_account: ${ secrets.GOOGLECLOUD_SERVICE_ACCOUNT }
    credentials_json: ${ secrets.GOOGLECLOUD_CREDENTIALS }
```

8. Now, we build and push the backend Docker image using `docker/build-push-action`, which builds and pushes an image to a Docker registry:

```
- name: Build and push backend image
  uses: docker/build-push-action@v4
  with:
    context: ./backend
```

```
file: ./backend/Dockerfile
push: true
tags: ${{ secrets.DOCKERHUB_USERNAME }}/blog-backend:latest
```

9. After pushing the Docker image for the backend, we can now deploy it on Cloud Run, using the **google-github-actions/deploy-cloudrun** action:

```
- id: deploy-backend
  name: Deploy backend
  uses: google-github-actions/deploy-cloudrun@v1
  with:
    service: blog-backend
    image: ${{ secrets.DOCKERHUB_USERNAME }}/blog-backend:latest
    region: ${{ secrets.GOOGLECLOUD_REGION }}
```

We gave this step the **deploy-backend** ID, as we need to use it to reference the backend URL to build the frontend image in the next step.

10. After building and deploying the backend, we build the frontend in a similar way, making sure to pass **VITE_BACKEND_URL** as **build-args**:

```
- name: Build and push frontend image
  uses: docker/build-push-action@v4
  with:
    context: .
    file: ./Dockerfile
    push: true
    tags: ${{ secrets.DOCKERHUB_USERNAME }}/blog-frontend:latest
    build-args: VITE_BACKEND_URL=${{ steps.deploy-backend.outputs.url }}/api/v1
```

11. Finally, we can deploy the frontend, giving this step the **deploy-frontend** ID, such that our environment URL can be set properly:

```
- id: deploy-frontend
  name: Deploy frontend
  uses: google-github-actions/deploy-cloudrun@v1
  with:
    service: blog-frontend
    image: ${{ secrets.DOCKERHUB_USERNAME }}/blog-frontend:latest
    region: ${{ secrets.GOOGLECLOUD_REGION }}
```

12. Save the file and commit and push your changes to the **main** branch. You will see **Deploy Blog Application** being triggered on GitHub Actions.

The following screenshot shows the result of our blog application being successfully deployed via GitHub Actions:

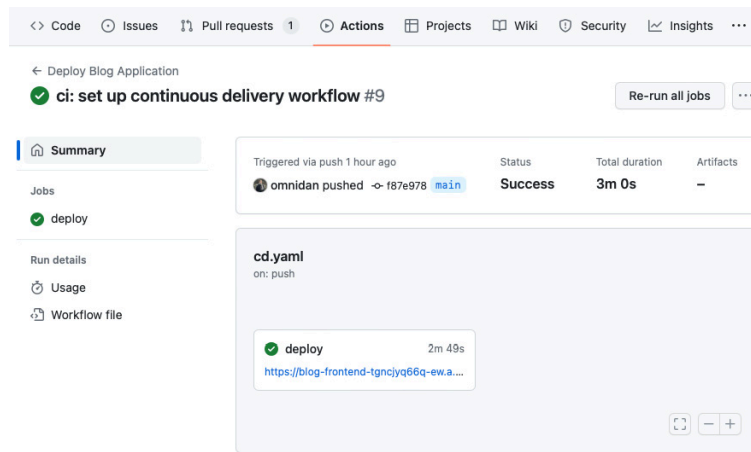


Figure 5.10 – A successful deployment of our full-stack application using GitHub Actions

You can click on the URL to open the deployed frontend and will see that it works the same way as the manually deployed version.

Congratulations! You have successfully automated the integration and deployment of your first full-stack application!

NOTE

In this book, we only created a single-stage deployment, deploying automatically directly to production. In a real-world application, you may want to define multiple stages. For example, CD could automatically deploy to a staging environment. Deploying to production could then be configured to require manual confirmation.

Summary

In this chapter, we started by learning how to create Docker images and how to instantiate local containers from them. Then, we automated this process by using Docker Compose. Next, we published our images on the Docker Hub registry to be able to deploy them on Google Cloud Run. We then manually deployed our full-stack application on Cloud Run. Finally, we learned how to set up CI/CD workflows with GitHub Actions to automate the running of the linter, tests, and deploying the blog application.

Up until now, everything in our application has been publicly accessible. With no user management, anyone can just create posts as any author. In the next chapter, [Chapter 6. Adding Authentication with JWT](#), we are going to learn how to implement user accounts and authentication in our full-stack blog application. We are going to learn what **JSON Web Tokens (JWTs)** are and implement multiple routes for logging in and signing up.